

LECTURE

APIs & fetch, *data from elsewhere.*

How a frontend asks a server for data, waits for the answer, and displays it in vanilla JavaScript and React.

API

JSON

fetch

async

useEffect

THE TARGET

By the end, remote data should feel reachable.

EXPLAIN

- ✓ What an API is.
- ✓ What JSON looks like.
- ✓ Why fetch takes time.
- ✓ Where fetch belongs in React.

BUILD

- ✓ One fetch call in plain JavaScript.
- ✓ A page update using fetched data.
- ✓ A loading state in React.
- ✓ A list rendered from API data.

one new layer at a time.

WHY FETCH?

Most useful apps need data that is not inside the file.

A weather app needs weather. A movie app needs movies. A store needs products. The frontend has to ask somewhere else.



fetch is how the browser asks.

PLAIN ENGLISH

An API is a URL that returns data instead of a webpage.

WEBPAGE URL

The browser receives HTML and shows a page for a person to read.

API URL

The browser receives data that code can read and use.

```
https://jsonplaceholder.typicode.com/users
```

open it in the browser first. no code yet.

DATA SHAPE

JSON looks like JavaScript data, but it is text.

```
{  
  "id": 1,  
  "name": "Leanne Graham",  
  "email": "Sincere@april.biz"  
}
```

JSON

JSON is a data format. It is common because almost every programming language can read it.

After `response.json()`, JavaScript can use it like an object or array.

SPECIFIC URLS

An endpoint is one specific API URL.

BASE URL

`https://jsonplaceholder.typicode.com`

USERS ENDPOINT

`/users`
A list of users.

ONE USER

`/users/1`
One user by id.

base URL

+

endpoint

=

full URL

THE TRIP

A fetch call is a request. The server sends a response.

1. code asks



2. network trip



3. server answers



4. code uses data

The slow part is the trip outside the computer. JavaScript has to wait for that answer.

TIME MATTERS

Normal code finishes now. Fetch finishes later.

SYNCHRONOUS

```
const name = "Maria";  
console.log(name);
```

ASYNCHRONOUS

```
fetch("/users/1");  
console.log("runs first");
```

fetch starts the work. it does not block the whole page.

FUTURE VALUE

A Promise means: “I do not have the result yet.”

1 Pending

The request is still happening.

2 Fulfilled

The request worked and has a result.

3 Rejected

Something failed while waiting.

We will use `async` and `await` so the code reads top to bottom.

TWO WAITS

Fetch has two async steps.

wait for response



wait for JSON



use data

```
const response = await fetch(url);  
const data = await response.json();
```

CONSOLE FIRST

Start by logging one user.

```
async function getUser() {  
  const response = await fetch("https://jsonplaceholder.typicode.com/users/1");  
  const data = await response.json();  
  
  console.log(data);  
}  
  
getUser();
```

first win: see the object before touching the page.

WHAT EACH PART MEANS

Every unfamiliar line has a job.

async	This function is allowed to use await.
await fetch(...)	Pause this function until the server responds.
response	The server answer. It is not the data yet.
response.json()	Convert the response body into JavaScript data.
data	The object or array the app can use.

DOM UPDATE

After the data arrives, update the page.

```
<h2 id="name">Loading...</h2>
```

```
async function getUser() {  
  const response = await fetch(url);  
  const user = await response.json();  
  
  document  
    .getElementById("name")  
    .textContent = user.name;  
}
```


ARRAY DATA

Some endpoints return arrays.

```
async function getUsers() {  
  const response = await fetch("https://jsonplaceholder.typicode.com/users");  
  const users = await response.json();  
  
  users.forEach(function(user) {  
    console.log(user.name);  
  });  
}
```

array from API, loop through it like any other array.

WHAT HAPPENED?

The server response includes a status code.

200

Success. The server found it.

201

Created. Usually after saving new data.

404

Not found. The URL or id may be wrong.

500

Server error. The backend failed.

the Network tab shows these clearly.

TRY / CATCH

When the request fails, show a useful path.

```
async function getUser() {  
  try {  
    const response = await fetch(url);  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.log("Something went wrong:", error);  
  }  
}
```

TRY

Attempt this code.

CATCH

If it fails, run this code instead.

NETWORK TAB

When fetch is confusing, check the request.

1 Request

Did the browser ask the URL you expected?

2 Status

Was it 200, 404, 500, or something else?

3 Response

What JSON came back from the server?

DevTools shows the trip.

WHERE DOES FETCH GO?

Fetch belongs inside useEffect.

`useEffect` runs code after the component appears on the screen. A network request reaches outside the component, so it is an effect.



ONE USER

Start with no user, then store the result.

```
import { useState, useEffect } from "react";

function UserCard() {
  const [user, setUser] = useState(null);

  useEffect(function() {
    async function fetchUser() {
      const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
      const data = await response.json();
      setUser(data);
    }

    fetchUser();
  }, []);
}
```

BEFORE DATA ARRIVES

React renders before the fetch finishes.

```
if (user === null) {  
  return <p>Loading...</p>;  
}  
  
return (  
  <div>  
    <h2>{user.name}</h2>  
    <p>{user.email}</p>  
  </div>  
);
```

WHY THIS MATTERS

At first, `user` is `null`. Without the check, `user.name` crashes because there is no user yet.

ARRAY STATE

For a list, start with an empty array.

```
function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(function() {
    async function fetchUsers() {
      const response = await fetch("https://jsonplaceholder.typicode.com/users");
      const data = await response.json();
      setUsers(data);
    }
    fetchUsers();
  }, []);

  return <ul>{users.map(function(user) {
    return <li key={user.id}>{user.name}</li>;
  })}</ul>;
}
```


COMMON CRASH

Choose the starting state to match the data.

ONE OBJECT LATER

Start with null.

```
const [user, setUser] =  
  useState(null);
```

ARRAY LATER

Start with [].

```
const [users, setUsers] =  
  useState([]);
```

you can map an empty array. you cannot map null.

KEEP NEARBY

The fetch checklist.

PLAIN JAVASCRIPT

- ✓ Use an async function.
- ✓ Await the fetch response.
- ✓ Await `response.json()`.
- ✓ Use the data after both awaits.

REACT

- ✓ Store data in state.
- ✓ Put fetch inside `useEffect`.
- ✓ Use `[]` to run once.
- ✓ Render loading before data exists.

ask, wait, convert, store, render.